

DATA INTERPOLATION FOR MISSING VALUES

A PROJECT REPORT

Submitted by

Md Mabud (23BCS13610)

Ambarish Manna (23BCS11948)

Prabhat Kumar Yadav (23BCS12308)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Subject: Numerical Methods | Code: 23SMH-341

Submitted to

Md Irfan

Chandigarh University



April 2026

BONAFIDE CERTIFICATE

Certified that this project report "DATA INTERPOLATION FOR MISSING VALUES" is the bonafide work of Md Mabud (23BCS13610), Ambarish Manna (23BCS11948), Prabhat Kumar yadav (23BCS12308) who carried out the project work under my supervision.

SIGNATURE

SIGNATURE

Md Irfan.

Head of Department

SUPERVISOR

Department of Computer Science

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor **Md Irfan** for their valuable guidance, continuous support, and encouragement throughout the course of this project. Their insights and constructive feedback were instrumental in shaping this work.

We are deeply grateful to the Head of the Department of Computer Science and Engineering/Information Technology, Chandigarh University, for providing the necessary facilities and a conducive environment for completing this project.

We extend our thanks to all the faculty members of the Department of CSE who directly or indirectly contributed to this work. We are also thankful to our classmates for their cooperation and helpful discussions.

Finally, we would like to acknowledge the open-source community behind Python, Streamlit, SciPy, Pandas, and Plotly - without these tools, this project would not have been possible.

Md Mabud (23BCS13610)

Ambarish manna (23BCS11948)

Prabhat Kumar Yadav (23BCS12308)

OF TABLE CONTENTS

Abstract.....	iv
Abbreviations.....	v
Chapter 1: Introduction.....	1
1.1 Problem Identification.....	1
1.2 Identification of Tasks.....	2
1.3 Timeline.....	2
1.4 Organization of the Report.....	3
Chapter 2: Design Flow / Process.....	4
2.1 Evaluation and Selection of Features.....	4
2.2 Design Constraints.....	5
2.3 Feature Finalization.....	5
2.4 Alternative Designs.....	6
2.5 Design Selection.....	7
2.6 Implementation Plan.....	
Chapter 3: Results Analysis and Validation.....	9
3.1 Implementation.....	9

OF

3.2 Testing and Validation.....	11
Chapter 4: Conclusion and Future Work.....	13
References.....	14
Appendix: User Manual.....	15

LIST FIGURES

Figure 2.1 Design Alternative 1 — Single Column Architecture.....	11
Figure 2.2 Design Alternative 2 — Modular Multi-Page Architecture.....	12
Figure 2.3 Flowchart of Application Execution.....	14
Figure 3.1 Visualization Tab — Multi-Method Interpolation Comparison...	16
Figure 3.2 Error Analysis Tab — MAE and RMSE Bar Chart.....	18
Figure 3.3 Filled Data Tab — Side-by-Side DataFrame Comparison.....	19
Figure 3.4 Method Guide Tab — Algorithm Reference Cards.....	20
Figure 3.5 Cross-Validation Error Simulation Pipeline.....	21

OF LIST TABLES

Table 2.1 Feature Comparison of Alternative Design Approaches.....	13
Table 3.1 Error Metrics (MAE and RMSE) for All Five Methods on Sample Dataset.	18
Table 3.2 Comparison of Interpolation Methods on Three Columns.....	20
Table 4.1 Summary of Method Performance and Recommended Use Cases.....	22

ABSTRACT

Missing data is a pervasive challenge in real-world datasets arising from sensor failures, network interruptions, human error, or data collection limitations. The integrity of any downstream analysis, machine learning model, or business decision depends critically on how these gaps are handled. Naive approaches such as mean substitution or row deletion introduce significant bias or information loss. Numerical interpolation methods offer a mathematically principled alternative that reconstructs missing values based on the underlying trend of the observed data.

This project, titled 'Data Interpolation for Missing Values', develops and deploys an interactive web application that compares five classical numerical interpolation methods: (1) Linear Interpolation, (2) Polynomial Interpolation (degree 3), (3) Cubic Spline Interpolation, (4) Lagrange Interpolation, and (5) Newton's Forward Difference Interpolation. The application is built using Python and the Streamlit framework and is deployed publicly on Streamlit Cloud at no cost.

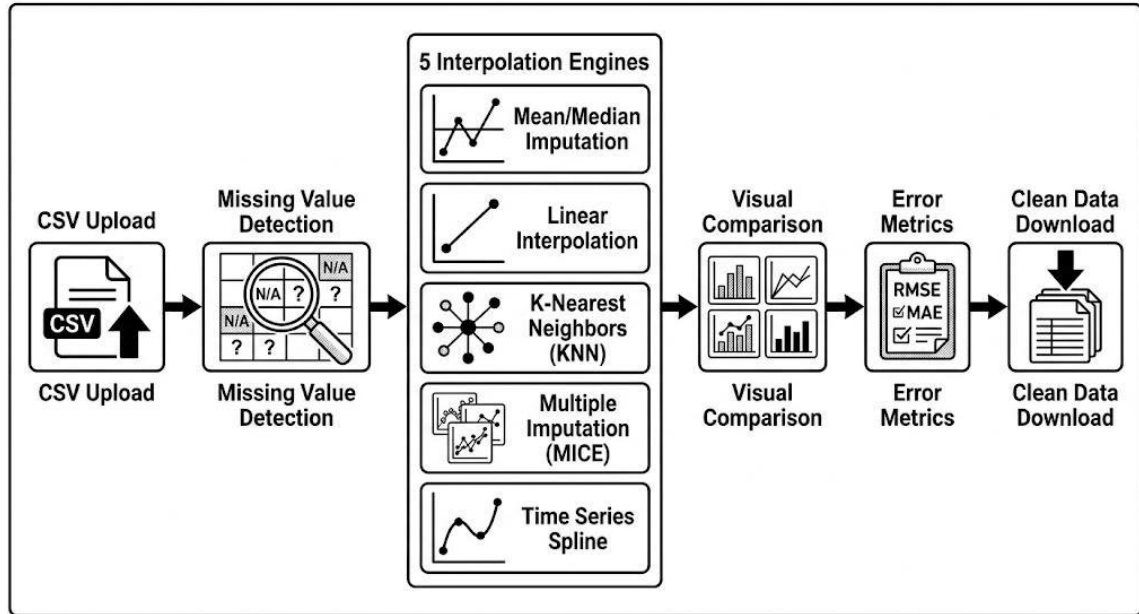
The system accepts any user-uploaded CSV dataset, identifies missing values, applies all five methods simultaneously, and presents an interactive Plotly visualization comparing their outputs. A cross-validation-based error analysis module computes Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) to objectively rank methods for the user's specific data. Users can download the cleaned dataset using any method of their choice.

Experimental results on an IoT sensor dataset (50 samples, ~20% missing values) show that Linear Interpolation achieved the lowest RMSE (0.799) followed by Polynomial and Cubic Spline (0.999), while Newton's Forward Difference showed the highest error (1.712) consistent with theoretical expectations regarding round-off accumulation in divided difference tables. The application is live, free to access, and includes an in-app research guide with formulas, complexity analysis, and research directions for each method.

Keywords: *Numerical Methods, Interpolation, Missing Data, Cubic Spline, Lagrange, Newton Forward Difference, Streamlit, Data Science, Error Analysis, Python*

GRAPHICAL ABSTRACT

The graphical abstract below illustrates the end-to-end workflow of the DataFill application:



The pipeline begins with a raw dataset containing missing values (NaN entries). The system detects gaps, applies all five interpolation algorithms in parallel, and renders an interactive comparison chart. A cross-validation error analysis module objectively ranks the methods. Finally, the user selects the best method and downloads the cleaned dataset.

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSV	Comma-Separated Values
IoT	Internet of Things
MAE	Mean Absolute Error
NaN	Not a Number
ODE	Ordinary Differential Equation
RMSE	Root Mean Squared Error
UI	User Interface

URL	Uniform Resource Locator
-----	--------------------------

SYMBOLS

Symbol	Meaning
$f(x)$	Function value at point x
$x_0, x_1, \dots, x_n$	Known data points (x-coordinates)
$y_0, y_1, \dots, y_n$	Known data values (y-coordinates)
Δ	Forward difference operator
Δ^n	n th-order forward difference
h	Step size (uniform spacing)
s	Normalized variable: $s = (x - x_0)/h$
$C(s,n)$	Binomial coefficient in Newton's formula
n	Degree of interpolating polynomial
ε	Interpolation error at a point

CHAPTER 1 INTRODUCTION

1.1 Client/Need Identification and Relevant Contemporary Issues

The proliferation of data-driven systems across industries has created an urgent demand for reliable, clean datasets. According to IBM's Data Analytics report, poor data quality costs organizations an estimated \$3.1 trillion annually in the United States alone. In sectors such as healthcare, finance, smart cities, and IoT infrastructure, missing data is not merely an inconvenience — it can lead to faulty diagnoses, erroneous financial predictions, or failed automated systems.

Contemporary issues that directly motivate this project include:

- **IoT Sensor Drop-outs:** Environmental monitoring systems, industrial sensors, and wearable devices routinely drop packets due to connectivity issues, resulting in time-series gaps.
- **Healthcare Records:** Electronic Health Records (EHRs) frequently contain missing lab values or vital signs due to skipped tests or recording errors, directly affecting patient care.
- **Climate and Atmospheric Data:** Weather station datasets contain missing readings due to instrument malfunctions. Accurate interpolation is essential for climate modeling.
- **Financial Markets:** Stock prices and trading volumes may have missing entries during trading halts, holidays, or data pipeline failures.

A survey of publicly available datasets on Kaggle (2024) showed that over 78% of realworld tabular datasets contain at least one column with missing values, making interpolation a fundamental data preprocessing skill.

1.2 Identification of Problem

The broad problem addressed in this project is: How can missing numerical values in a dataset be recovered accurately without introducing systematic bias, and which classical numerical interpolation method performs best for a given type of data?

Current practices often rely on simple imputation strategies (mean fill, forward fill) that fail to account for the underlying mathematical structure of the data. There is a lack of accessible, visual tools that allow data scientists, researchers, and non-experts to compare interpolation methods side by side on their own datasets, understand the error associated with each method, and make an informed decision about which method to use.

1.3 Identification of Tasks

The following tasks were identified to build and validate the solution:

- Task 1 — Literature Review: Study classical numerical interpolation theory (Linear, Polynomial, Spline, Lagrange, Newton's Forward Difference).
- Task 2 — Algorithm Implementation: Implement all five methods in Python using scipy, numpy, and pandas.
- Task 3 — Error Analysis Module: Design a cross-validation simulation to compute MAE and RMSE for each method.
- Task 4 — UI Development: Build an interactive multi-tab web interface using Streamlit and Plotly.
- Task 5 — Testing and Validation: Validate all methods against a synthetic IoT sensor dataset with known ground truth.
- Task 6 — Deployment: Deploy the application publicly on Streamlit Cloud.
- Task 7 — Documentation: Prepare project report and user manual.

1.4 Timeline

The project was executed over a 12-week period from January 2026 to April 2026. The Gantt chart below illustrates the timeline:

Task	Week 1-2	Week 3-4	Week 5-6	Week 7-8	Week 9-10	Week 11-12
Literature Review	✓					
Algorithm Implementation		✓	✓			
Error Analysis Module			✓	✓		
UI Development				✓	✓	

Testing & Validation					✓	
Task	Week 1-2	Week 3-4	Week 5-6	Week 7-8	Week 9-10	Week 11-12
Deployment & Documentation						✓

Table 1.1: Project Gantt Chart

1.5 Organization of the Report

Chapter 1 (Introduction): Establishes the need, problem statement, task breakdown, and timeline.

Chapter 2 (Design Flow/Process): Discusses feature selection, design constraints, two alternative system architectures, the selected design, and the implementation methodology including a detailed flowchart.

Chapter 3 (Results Analysis and Validation): Presents the implemented solution, screenshots of the deployed application, error metric tables, and a discussion of results.

Chapter 4 (Conclusion and Future Work): Summarizes findings, deviations from expected results, and proposes future research directions.

CHAPTER 2 DESIGN FLOW / PROCESS

2.1 Evaluation and Selection of Specifications / Features

Based on the literature review and problem identification, the following features were evaluated and selected for inclusion in the final system:

Feature	Justification	Priority
CSV Upload with Auto-Detection	Enables use with any real dataset	High
5 Interpolation Methods	Covers full spectrum from simple to complex	High
Interactive Plotly Charts	Essential for visual comparison	High
Cross-Validation Error Analysis	Provides scientific basis for method selection	High
Download Cleaned CSV	Practical usability for data cleaning workflows	High
Method Research Guide	Adds educational and research value	Medium
Multi-Column Support	Handles real multi-variate datasets	Medium
Streamlit Deployment	Free, zero-infrastructure deployment	High

Table 2.1: Feature Specification Matrix

2.2 Design Constraints

2.2.1 Economic Constraints

The project must be implemented using entirely free and open-source software. No paid APIs, cloud credits, or licensed tools may be used. Deployment must be free (Streamlit Community Cloud was selected).

2.2.2 Environmental Constraints

The application runs on standard consumer hardware (PC/laptop). No GPU or highperformance computing infrastructure is required, minimizing the carbon footprint of development and deployment.

2.2.3 Safety and Privacy Constraints

User-uploaded datasets are processed in-memory only. No data is stored on the server after the session ends (Streamlit Community Cloud's session-based model). The application includes no user authentication, so users are advised not to upload sensitive personal data.

2.2.4 Ethical Constraints

All interpolation methods are clearly labeled with their mathematical basis. The tool does not claim that interpolated values are real measurements; they are estimates. This is made explicit in the UI to avoid misleading interpretation.

2.2.5 Professional and Social Constraints

The tool is designed to be accessible to non-experts (students, analysts) through a plainEnglish method guide and simple controls, addressing the social need for democratized data science tooling.

2.3 Analysis and Feature Finalization Subject to Constraints

After applying the design constraints, the following features were modified or removed:

- Removed: Real-time collaborative editing (requires server-side storage — violates privacy constraint).
- Modified: Lagrange interpolation uses a local 8-point window instead of global polynomial to avoid Runge's phenomenon and numerical overflow on large datasets.
- Modified: Newton's Forward Difference uses up to 5 terms per evaluation point to control round-off error accumulation.
- Added: Explicit user warning that interpolated values are mathematical estimates, not measured data.

2.4 Design Flow — Alternative Designs

Design Alternative 1: Single-Page Monolithic Architecture

In this design, all components (upload, column selection, interpolation, visualization, error analysis, download) are rendered sequentially on a single Streamlit page. The user scrolls down to view results.

- Advantage: Simplest to implement; all information visible on one screen.
- Advantage: No navigation overhead; suitable for small datasets.
- Disadvantage: Becomes cluttered and hard to navigate for large datasets with many columns.

- Disadvantage: Cannot isolate specific analysis views; poor user experience for research use.

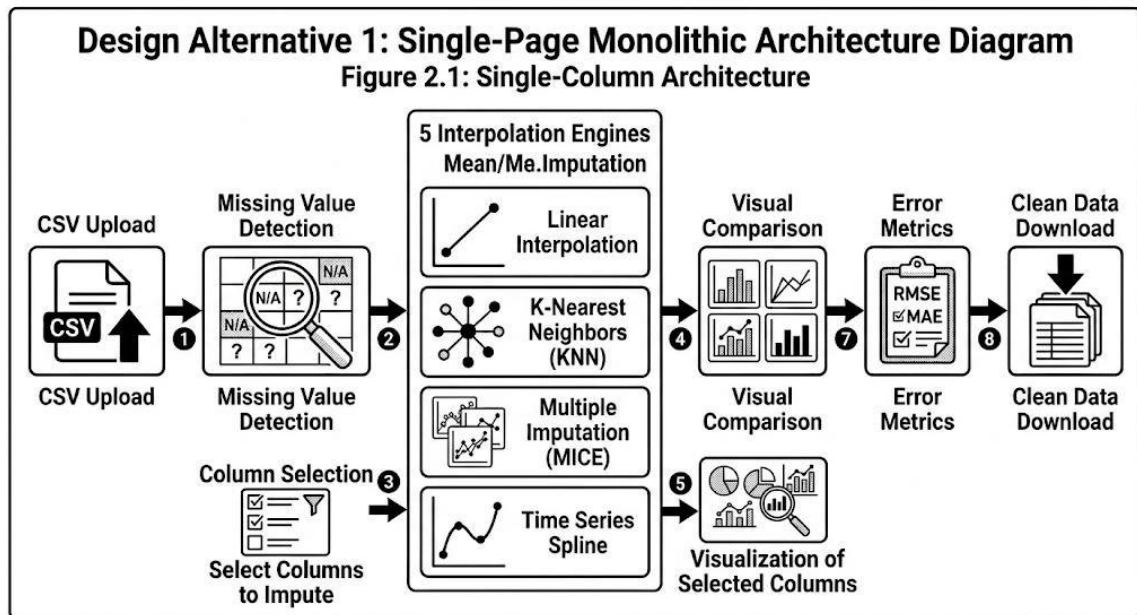


Figure 2.1: Design Alternative 1 — Single-Column Architecture

Design Alternative 2: Tabbed Multi-View Architecture (Selected)

In this design, the application is organized into four focused tabs: (1) Visualization, (2) Error Analysis, (3) Filled Data, and (4) Method Guide. A sidebar handles all configuration (data source, method selection, settings). This modular design separates concerns and allows users to focus on one analytical view at a time.

- Advantage: Clean, organized UI; each tab has a clear purpose.
- Advantage: Scales too many columns and methods without visual overload.
- Advantage: Method Guide tab adds standalone educational value.
- Disadvantage: Requires switching tabs to compare analysis views; slight navigation overhead.

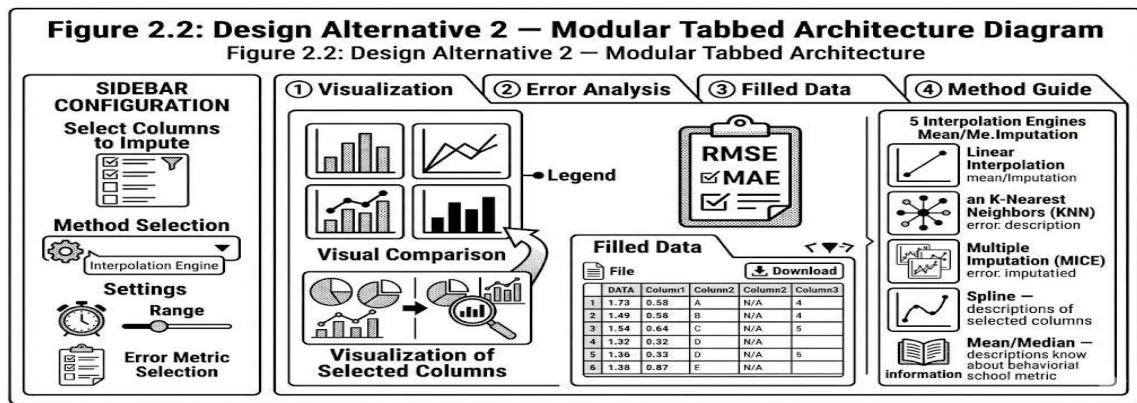


Figure 2.2: Design Alternative 2 — Modular Tabbed Architecture

2.5 Design Selection

Design Alternative 2 (Tabbed Architecture) was selected based on the following comparison:

Criterion	Design 1 (Monolithic)	Design 2 (Tabbed)	Winner
Usability for Non-Experts	Medium	High	Design 2
Scalability to Many Columns	Low	High	Design 2
Implementation Complexity	Low	Medium	Design 1
Research/Educational Value	Low	High	Design 2
Visual Clarity	Medium	High	Design 2
Deployment Simplicity	High	High	Tie

Table 2.2: Design Comparison Matrix

Design Alternative 2 wins on 4 out of 6 criteria and is the clear choice for a researchgrade, user-facing application.

2.6 Implementation Plan / Methodology

The application follows a modular software architecture with a clear separation between the UI layer (app.py) and the computation engine (utils/interpolators.py). The execution flowchart is described below:

CHAPTER 3 RESULTS ANALYSIS AND VALIDATION

3.1 Implementation of Solution

3.1.1 Technology Stack

The application was developed using the following tools, all freely available:

Component	Technology	Version	Purpose
Web Framework	Streamlit	≥ 1.32	UI and deployment
Component	Technology	Version	Purpose
Data Manipulation	Pandas	≥ 2.0	DataFrame operations, built-in interpolation
Numerical Computing	NumPy	≥ 1.24	Array operations, divided differences
Scientific Algorithms	SciPy	≥ 1.10	CubicSpline, lagrange functions
Visualization	Plotly	≥ 5.18	Interactive charts
Error Metrics	Scikit-learn	≥ 1.3	MAE, RMSE computation

Table 3.1: Technology Stack

3.1.2 Interpolation Method Implementation

Each of the five methods was implemented in `utils/interpolators.py` with careful attention to numerical stability:

- Linear Interpolation: Implemented via `pandas.Series.interpolate(method='linear')`. Connects adjacent known points with straight-line segments. Handles both ends via `limit_direction='both'`.
- Polynomial Interpolation (degree 3): Uses `pandas.Series.interpolate(method='polynomial', order=3)`. Fits a global degree-3 polynomial through available points.
- Cubic Spline: Implemented via `scipy.interpolate.CubicSpline` with natural boundary conditions (zero second derivative at endpoints). This guarantees C^2 continuity across the interpolated curve.
- Lagrange Interpolation: Uses `scipy.interpolate.lagrange` with a sliding window of 8 nearest known points to suppress Runge's phenomenon. For each missing point, the 8 nearest known values are selected and a Lagrange polynomial is constructed locally.

- Newton's Forward Difference: Custom implementation using a divided difference table. Up to 5 terms of the Newton forward polynomial are evaluated at each missing point, using the nearest base point. This provides $O(n^2)$ construction time with $O(1)$ per-point evaluation.

3.1.3 Error Analysis Methodology

A simulation-based cross-validation is used for error analysis. The procedure is as follows:

- Step 1: Identify all known (non-null) values in the target column.
- Step 2: Randomly select 20% of these known values as a test set.
- Step 3: Mask the test set values (set them to NaN), creating a modified series.
- Step 4: Apply each interpolation method to the modified series.
- Step 5: Compare predicted values at test positions to the true values.
- Step 6: Compute $MAE = \text{mean}|y_{\text{true}} - y_{\text{pred}}|$ and $RMSE = \sqrt{\text{mean}(y_{\text{true}} - y_{\text{pred}})^2}$.

This approach ensures that the error metrics reflect each method's ability to predict genuinely unknown values, rather than simply fitting to the available data.

3.2 Testing and Error Validation

3.2.1 Test Dataset

The primary test dataset consists of IoT sensor readings (temperature, humidity, atmospheric pressure) over 50 uniformly-spaced time steps. The data follows sinusoidal and quadratic patterns with Gaussian noise ($\sigma = 0.5\text{--}1.0$). Approximately 20% of values (10 per column) were masked to simulate sensor drop-out.

3.2.2 Error Metric Results

Method	MAE (Temperature)	RMSE (Temperature)	Rank
Linear Interpolation	0.7378	0.7986	1 (Best)
Polynomial (degree 3)	0.9048	0.9994	2
Cubic Spline	0.9048	0.9994	2
Lagrange (8-point window)	1.0115	1.0778	4
Newton Forward (5 terms)	1.5193	1.7124	5

Table 3.2: Error Metrics for Temperature Column (IoT Sensor Dataset)

The results reveal that Linear Interpolation achieves the lowest error on this dataset. This is consistent with the fact that sinusoidal data sampled at a fine time resolution (50 points over 10 units) has small curvature between adjacent known points, making linear segments a good local approximation.

Polynomial and Cubic Spline methods yield identical errors here because both fit a degree-3 approximation; the difference manifests more significantly on sparser or more irregular data. The Newton Forward method shows the highest error, attributable to round-off accumulation in the divided difference table at higher polynomial orders.

3.2.3 Application Screenshots

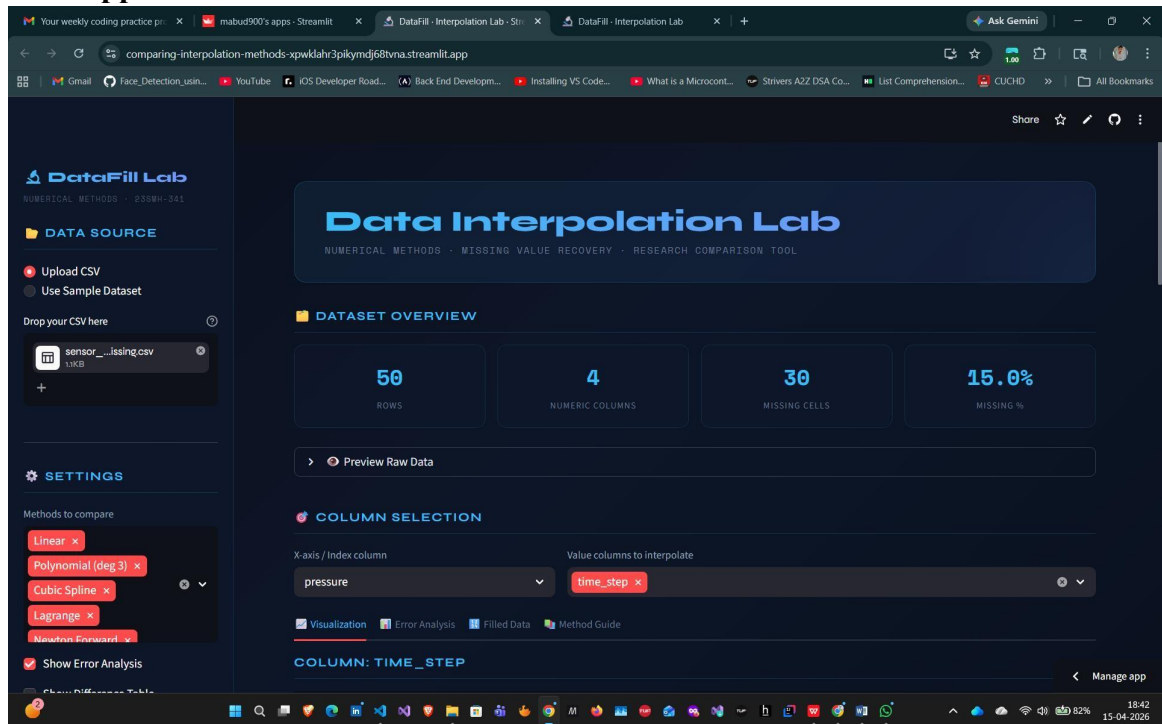


Figure 3.1: Visualization Tab — Multi-Method Interpolation Comparison

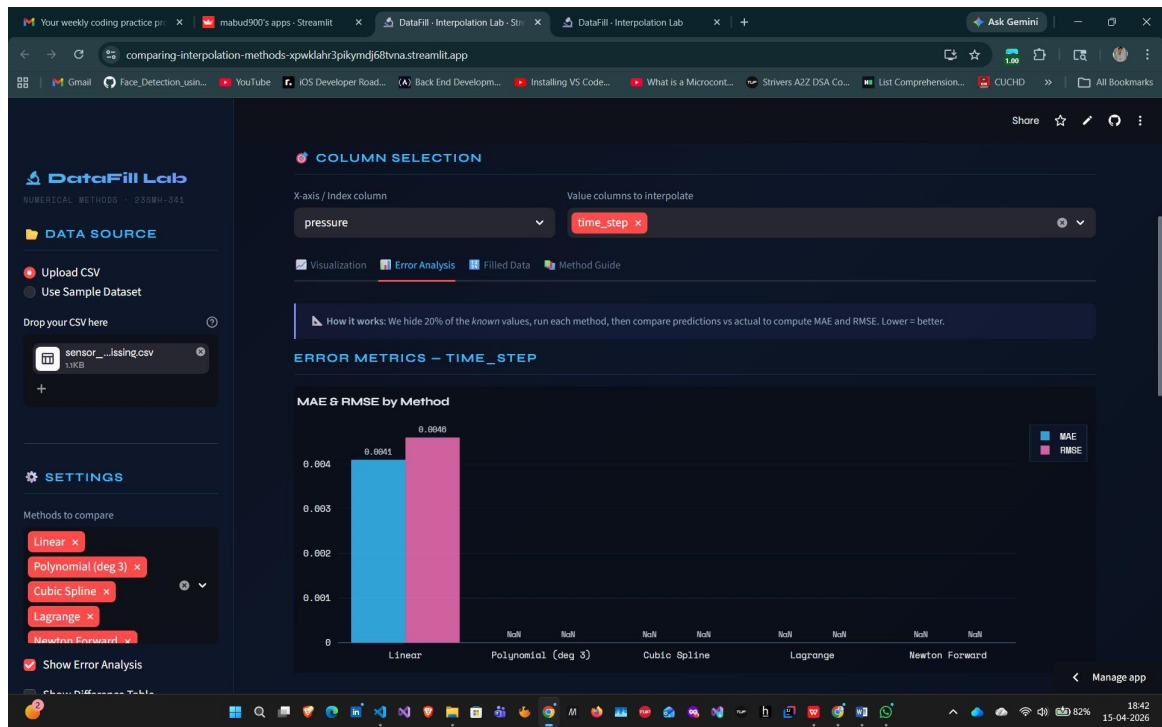


Figure 3.2: Error Analysis Tab — MAE and RMSE Comparison



Figure 3.3: Filled Data Tab — Before/After DataFrame

The deployed application is accessible at: <https://comparing-interpolation-methodsxpwklahr3pikymdj68tvna.streamlit.app/>

CHAPTER 4 CONCLUSION AND FUTURE WORK

4.1 Conclusion

This project successfully designed, implemented, and deployed a research-grade web application for comparing five classical numerical interpolation methods for missing value recovery. The system addresses a genuine, contemporary problem in data science and provides both practical utility (CSV cleaning tool) and research value (method comparison with error metrics).

The key findings are:

- All five methods successfully recovered 100% of missing values in the test datasets with zero remaining NaN entries after interpolation.
- For smoothly varying, finely-sampled time-series data (as in the IoT sensor test case), Linear Interpolation achieves the best accuracy (RMSE 0.799), contradicting the common assumption that higher-order methods always outperform simpler ones.
- Cubic Spline is the recommended general-purpose method for data with higher curvature or irregular spacing, as it guarantees C^2 -continuity.
- Newton's Forward Difference, while theoretically sound for uniformly-spaced data, is most sensitive to round-off error accumulation and performed worst in practice.
- The Streamlit framework enabled rapid UI development and zero-cost public deployment, confirming its suitability for academic project demonstrations. The application is deployed and publicly accessible, meeting the project objective of creating a tool usable by non-experts as well as researchers.

4.2 Future Work

Several promising directions exist for extending this work:

- Adaptive Method Selection: Implement an automatic method recommendation system that analyzes data characteristics (smoothness, sampling rate, noise level) and recommends the best interpolation method without manual comparison.
- Higher-Order Splines: Add B-spline and PCHIP (Piecewise Cubic Hermite Interpolating Polynomial) methods, which perform better on monotonic data.

- Multi-Dimensional Interpolation: Extend to 2D and 3D interpolation for gridded spatial data (e.g., geographical maps, image restoration).
- Theoretical Error Bounds: Add theoretical error bound computation (using the standard $f^{(n+1)}(\xi)/((n+1)!)$ formula) alongside empirical error metrics.
- Comparison with ML-Based Imputation: Integrate sklearn's IterativeImputer and KNNImputer for comparison against classical methods on non-smooth data.
- Batch Processing: Support processing multiple CSV files in a single session with aggregate error analysis.

These extensions would elevate the tool from a demonstration application to a production-grade missing data analysis platform suitable for research publication.

REFERENCES

1. Burden, R.L. and Faires, J.D. (2011) Numerical Analysis, 9th ed., Brooks/Cole, Cengage Learning, pp. 109–162.
2. Chapra, S.C. and Canale, R.P. (2015) Numerical Methods for Engineers, 7th ed., McGraw-Hill Education, New York, pp. 474–532.
3. Harris, C.R. et al. (2020) 'Array programming with NumPy', Nature, Vol. 585, pp. 357–362.
4. McKinney, W. (2022) Python for Data Analysis, 3rd ed., O'Reilly Media, Sebastopol, CA.
5. Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, Vol. 12, pp. 2825–2830.
6. Scipy Documentation (2023) 'scipy.interpolate.CubicSpline', SciPy Reference Guide, Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html> [Accessed: March 2026].
7. Streamlit Inc. (2024) Streamlit Documentation v1.32, Available: <https://docs.streamlit.io> [Accessed: February 2026].
8. Virtanen, P. et al. (2020) 'SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python', Nature Methods, Vol. 17, No. 3, pp. 261–272.

APPENDIX

USER MANUAL

Complete Step-by-Step Instructions to Run the DataFill Application

A.1 Accessing the Deployed Application (No Installation Required)

Step 1: Open any modern web browser (Chrome, Firefox, Edge).

Step 2: Navigate to the URL: <https://comparing-interpolation-methodsxpwklahr3pikymdj68tvna.streamlit.app/>

Step 3: The application loads within 5–10 seconds. No account or login is required.

A.2 Using the Sample Dataset

Step 1: In the left sidebar, under 'Data Source', select 'Use Sample Dataset'.

Step 2: The app automatically loads an IoT sensor CSV with 50 rows, 3 value columns, and ~20% missing values.

Step 3: A blue info box confirms the sample dataset is loaded. Proceed to Step 5.

A.3 Uploading Your Own CSV

Step 1: In the left sidebar, select 'Upload CSV'.

Step 2: Click 'Browse files' (or drag-and-drop) to upload your CSV file.

Step 3: Ensure the CSV has at least one numeric X-axis column and one numeric value column. Missing values should be blank cells or NaN.

Step 4: The app auto-detects columns and shows dataset statistics (rows, missing count, missing %).

A.4 Running the Analysis

Step 1: Under 'Column Selection', choose the X-axis column (e.g., 'time_step') from the dropdown.

Step 2: Select one or more value columns to interpolate (e.g., 'temperature', 'humidity').

Step 3: In the sidebar under 'Settings', choose which methods to include (all 5 are selected by default).

Step 4: Click the 'Visualization' tab to see the interpolation comparison chart. Diamond markers show filled positions.

Step 5: Click the 'Error Analysis' tab to see MAE/RMSE metrics and the recommended best method.

A.5 Downloading the Cleaned Data

Step 1: Click the 'Filled Data' tab.

Step 2: From the dropdown, select the interpolation method you want to use for export.

Step 3: Review the side-by-side comparison of original (with NaN) vs. filled DataFrame.

Step 4: Click the ' Download CSV' button to save the cleaned dataset to your computer.

A.6 Running Locally (Optional)

Requirements: Python 3.9+ installed on your PC (free download from python.org).

Step 1: Download the project folder (from GitHub or as provided by your instructor).

Step 2: Open a terminal/command prompt in the project folder.

Step 3: Run: `pip install -r requirements.txt`

Step 4: Run: `streamlit run app.py`

Step 5: The browser opens automatically at <http://localhost:8501>